

Lab 4 Tutorial: React Context, Redux, TypeScript, and Advanced React Native Concepts

This comprehensive tutorial covers advanced React Native concepts for building complex mobile applications:

1. **React Context API** - Sharing state across components without prop drilling
 2. **Nested Contexts** - Multiple contexts for different app concerns
 3. **Data Passing with Context** - Sharing data across screens and components
 4. **Brief Introduction to Redux** - State management alternative
 5. **Platform-Specific Code** - Handling iOS and Android differences
 6. **Combining All Contexts** - Complete example with all contexts together
 7. **Brief Introduction to TypeScript** - Type safety in React Native
 8. **React Native DevTools** - Debugging and development tools
-

1. React Context API

React Context provides a way to share data across multiple components without passing props through every level of the component tree (prop drilling).

What is Context?

Context allows you to share data that can be considered "global" for a tree of React components, such as:

- User authentication state
- Theme preferences
- Language settings
- Shopping cart data
- User profile information

Why Use Context?

Problems with Prop Drilling:

```
// Without Context - passing props through many levels
function App() {
  const [userEmail, setUserEmail] = useState('');
  return <Parent email={userEmail} setEmail={setUserEmail} />;
}

function Parent({ email, setEmail }) {
  return <Child email={email} setEmail={setEmail} />;
}

function Child({ email, setEmail }) {
  return <GrandChild email={email} setEmail={setEmail} />;
}
```

```
function GrandChild({ email, setEmail }) {  
  return <Text>{email}</Text>; // Finally using the data  
}
```

Solution with Context:

```
// contexts/UserContext.js  
import { createContext, useContext, useState } from 'react';  
  
const UserContext = createContext();  
  
export const useUser = () => {  
  const context = useContext(UserContext);  
  if (!context) {  
    throw new Error('useUser must be used within UserContext.Provider');  
  }  
  return context;  
};  
  
export const UserProvider = ({ children }) => {  
  const [userEmail, setUserEmail] = useState('');  
  return (  
    <UserContext.Provider value={{ email: userEmail, setEmail: setUserEmail }}>  
      {children}  
    </UserContext.Provider>  
  );  
};
```

```
// App.js  
import { UserProvider } from './contexts/UserContext';  
  
function App() {  
  return (  
    <UserProvider>  
      <Parent />  
    </UserProvider>  
  );  
}  
  
function Parent() {  
  return <Child />;  
}  
  
function Child() {  
  return <GrandChild />;  
}  
  
function GrandChild() {  
  const { email } = useUser(); // Direct access!
```

```
    return <Text>{email}</Text>;  
  }
```

Creating a Context

Step 1: Create the Context in a Separate File

```
// contexts/UserContext.js  
import { createContext, useContext, useState } from 'react';  
  
const UserContext = createContext();  
  
export const useUser = () => {  
  const context = useContext(UserContext);  
  if (!context) {  
    throw new Error('useUser must be used within a UserContext.Provider');  
  }  
  return context;  
};  
  
export const UserProvider = ({ children }) => {  
  const [userEmail, setUserEmail] = useState('');  
  
  return (  
    <UserContext.Provider value={{ email: userEmail, setEmail: setUserEmail }}>  
      {children}  
    </UserContext.Provider>  
  );  
};
```

Step 2: Use the Context Provider in App.js

```
// App.js  
import { UserProvider } from './contexts/UserContext';  
  
export default function App() {  
  return (  
    <UserProvider>  
      { /* All child components can access userEmail and setUserEmail */ }  
      <YourApp />  
    </UserProvider>  
  );  
}
```

Step 3: Consume the Context in Components

```
// screens/ProfileScreen.js
import { useUser } from '../contexts/UserContext';

function ProfileScreen() {
  const { email, setEmail } = useUser();

  return (
    <View>
      <Text>Logged in as: {email}</Text>
      <Button
        title="Logout"
        onPress={() => setEmail('')}
      />
    </View>
  );
}
```

Complete Context Example

```
// contexts/UserContext.js
import { createContext, useContext, useState } from 'react';

const UserContext = createContext();

export const useUser = () => {
  const context = useContext(UserContext);
  if (!context) {
    throw new Error('useUser must be used within UserContext.Provider');
  }
  return context;
};

export const UserProvider = ({ children }) => {
  const [userEmail, setUserEmail] = useState('');

  return (
    <UserContext.Provider value={{ email: userEmail, setEmail: setUserEmail }}>
      {children}
    </UserContext.Provider>
  );
};
```

```
// components/ProfileDisplay.js
import { View, Text, Button, StyleSheet } from 'react-native';
import { useUser } from '../contexts/UserContext';

export default function ProfileDisplay() {
  const { email, setEmail } = useUser();
```

```

    return (
      <View style={styles.container}>
        <Text style={styles.text}>Current User: {email || 'Not logged in'}</Text>
        <Button
          title="Logout"
          onPress={() => setEmail('')}
        />
      </View>
    );
  }

  const styles = StyleSheet.create({
    container: {
      marginTop: 20,
      padding: 20,
      backgroundColor: '#f0f0f0',
      borderRadius: 8,
    },
    text: {
      fontSize: 16,
      marginBottom: 10,
    },
  });

```

```

// App.js
import { View, Text, StyleSheet } from 'react-native';
import { UserProvider } from '../contexts/UserContext';
import ProfileDisplay from '../components/ProfileDisplay';

export default function App() {
  return (
    <UserProvider>
      <View style={styles.appContainer}>
        <Text style={styles.title}>Context Example</Text>
        <ProfileDisplay />
      </View>
    </UserProvider>
  );
}

const styles = StyleSheet.create({
  appContainer: {
    flex: 1,
    padding: 20,
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
  },
});

```

```
    },  
  });  
};
```

2. Nested Contexts - Multiple Contexts for Different Concerns

In real-world applications, you'll often need multiple contexts for different concerns. Here are practical examples:

Example 1: User Authentication State Context

```
// contexts/AuthContext.js  
import { createContext, useContext, useState, useEffect } from 'react';  
  
const AuthContext = createContext();  
  
export const useAuth = () => {  
  const context = useContext(AuthContext);  
  if (!context) {  
    throw new Error('useAuth must be used within AuthContext.Provider');  
  }  
  return context;  
};  
  
export const AuthProvider = ({ children }) => {  
  const [user, setUser] = useState(null);  
  const [isLoading, setIsLoading] = useState(true);  
  
  useEffect(() => {  
    // Initialize loading state  
    setIsLoading(false);  
  }, []);  
  
  const login = async (email, password) => {  
    try {  
      // Simulate API call  
      const userData = { id: 1, email, name: 'John Doe' };  
      setUser(userData);  
      return { success: true };  
    } catch (error) {  
      return { success: false, error: error.message };  
    }  
  };  
  
  const logout = () => {  
    setUser(null);  
  };  
  
  const value = {  
    user,  
    isLoading,  
    login,  
    logout  
  };  
};
```

```

    login,
    logout,
    isAuthenticated: !!user,
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
};

// Usage in component
function LoginScreen() {
  const { login, isAuthenticated } = useAuth();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const handleLogin = async () => {
    const result = await login(email, password);
    if (result.success) {
      console.log('Logged in successfully!');
    }
  };

  return (
    <View>
      <TextInput value={email} onChangeText={setEmail} placeholder="Email" />
      <TextInput value={password} onChangeText={setPassword}
placeholder="Password" secureTextEntry />
      <Button title="Login" onPress={handleLogin} />
    </View>
  );
}

```

```

// App.js - Using AuthContext
import { AuthProvider } from './contexts/AuthContext';
import LoginScreen from './screens/LoginScreen';

export default function App() {
  return (
    <AuthProvider>
      <LoginScreen />
    </AuthProvider>
  );
}

```

Example 2: Theme Context (Dark/Light Mode)

```

// contexts/ThemeContext.js
import { createContext, useContext, useState, useEffect } from 'react';
import { useColorScheme } from 'react-native';

```

```
const ThemeContext = createContext();

export const useTheme = () => {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error('useTheme must be used within ThemeContext.Provider');
  }
  return context;
};

export const ThemeProvider = ({ children }) => {
  const systemTheme = useColorScheme();
  const [theme, setTheme] = useState(systemTheme || 'light');
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    setIsLoading(false);
  }, []);

  const toggleTheme = () => {
    const newTheme = theme === 'light' ? 'dark' : 'light';
    setTheme(newTheme);
  };

  const colors = {
    light: {
      background: '#FFFFFF',
      text: '#000000',
      primary: '#007AFF',
      secondary: '#5856D6',
    },
    dark: {
      background: '#000000',
      text: '#FFFFFF',
      primary: '#0A84FF',
      secondary: '#5E5CE6',
    },
  };

  const value = {
    theme,
    colors: colors[theme],
    toggleTheme,
    isLoading,
  };

  return <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>;
};

// Usage in component
function HomeScreen() {
  const { colors, toggleTheme } = useTheme();

  return (
```

```
    <View style={{ flex: 1, backgroundColor: colors.background }}>
      <Text style={{ color: colors.text }}>Welcome!</Text>
      <Button title="Toggle Theme" onPress={toggleTheme} />
    </View>
  );
}
```

```
// App.js - Using ThemeContext
import { ThemeProvider } from './contexts/ThemeContext';
import HomeScreen from './screens/HomeScreen';

export default function App() {
  return (
    <ThemeProvider>
      <HomeScreen />
    </ThemeProvider>
  );
}
```

Example 3: Language Preferences Context

```
// contexts/LanguageContext.js
import { createContext, useContext, useState, useEffect } from 'react';
import * as Localization from 'expo-localization';

const LanguageContext = createContext();

export const useLanguage = () => {
  const context = useContext(LanguageContext);
  if (!context) {
    throw new Error('useLanguage must be used within LanguageContext.Provider');
  }
  return context;
};

const translations = {
  en: {
    welcome: 'Welcome',
    login: 'Login',
    logout: 'Logout',
    settings: 'Settings',
  },
  es: {
    welcome: 'Bienvenido',
    login: 'Iniciar sesión',
    logout: 'Cerrar sesión',
    settings: 'Configuración',
  },
  fr: {
```

```

    welcome: 'Bienvenue',
    login: 'Connexion',
    logout: 'Déconnexion',
    settings: 'Paramètres',
  },
};

export const LanguageProvider = ({ children }) => {
  const [language, setLanguage] = useState('en');
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    // Use device language on initialization
    const deviceLanguage = Localization.locale.split('-')[0];
    setLanguage(translations[deviceLanguage] ? deviceLanguage : 'en');
    setIsLoading(false);
  }, []);

  const changeLanguage = (lang) => {
    setLanguage(lang);
  };

  const t = (key) => {
    return translations[language]?.[key] || key;
  };

  const value = {
    language,
    changeLanguage,
    t,
    isLoading,
  };

  return <LanguageContext.Provider value={value}>{children}
</LanguageContext.Provider>;
};

// Usage in component
function WelcomeScreen() {
  const { t, changeLanguage } = useLanguage();

  return (
    <View>
      <Text>{t('welcome')}</Text>
      <Button title="English" onPress={() => changeLanguage('en')} />
      <Button title="Español" onPress={() => changeLanguage('es')} />
      <Button title="Français" onPress={() => changeLanguage('fr')} />
    </View>
  );
}

```

```
// App.js - Using LanguageContext
import { LanguageProvider } from './contexts/LanguageContext';
import WelcomeScreen from './screens/WelcomeScreen';

export default function App() {
  return (
    <LanguageProvider>
      <WelcomeScreen />
    </LanguageProvider>
  );
}
```

Example 4: Shopping Cart Context

```
// contexts/CartContext.js
import { createContext, useContext, useState, useCallback } from 'react';

const CartContext = createContext();

export const useCart = () => {
  const context = useContext(CartContext);
  if (!context) {
    throw new Error('useCart must be used within CartContext.Provider');
  }
  return context;
};

export const CartProvider = ({ children }) => {
  const [items, setItems] = useState([]);

  const addItem = useCallback((product) => {
    setItems((prevItems) => {
      const existingItem = prevItems.find((item) => item.id === product.id);
      if (existingItem) {
        return prevItems.map((item) =>
          item.id === product.id
            ? { ...item, quantity: item.quantity + 1 }
            : item
        );
      }
      return [...prevItems, { ...product, quantity: 1 }];
    });
  }, []);

  const removeItem = useCallback((productId) => {
    setItems((prevItems) => prevItems.filter((item) => item.id !== productId));
  }, []);

  const updateQuantity = useCallback((productId, quantity) => {
    if (quantity <= 0) {

```

```

        removeItem(productId);
        return;
    }
    setItems((prevItems) =>
        prevItems.map((item) =>
            item.id === productId ? { ...item, quantity } : item
        )
    );
}, [removeItem]);

const clearCart = useCallback(() => {
    setItems([]);
}, []);

const getTotalPrice = useCallback(() => {
    return items.reduce((total, item) => total + item.price * item.quantity, 0);
}, [items]);

const getTotalItems = useCallback(() => {
    return items.reduce((total, item) => total + item.quantity, 0);
}, [items]);

const value = {
    items,
    addItem,
    removeItem,
    updateQuantity,
    clearCart,
    getTotalPrice,
    getTotalItems,
};

return <CartContext.Provider value={value}>{children}</CartContext.Provider>;
};

// Usage in component
function ProductCard({ product }) {
    const { addItem } = useCart();

    return (
        <View>
            <Text>{product.name}</Text>
            <Text>${product.price}</Text>
            <Button title="Add to Cart" onPress={() => addItem(product)} />
        </View>
    );
}

function CartScreen() {
    const { items, removeItem, updateQuantity, getTotalPrice, clearCart } =
    useCart();

    return (
        <View>

```

```

    {items.map((item) => (
      <View key={item.id}>
        <Text>{item.name}</Text>
        <Text>Quantity: {item.quantity}</Text>
        <Text>Price: ${item.price * item.quantity}</Text>
        <Button title="Remove" onPress={() => removeItem(item.id)} />
        <Button title="+" onPress={() => updateQuantity(item.id, item.quantity +
1)} />
        <Button title="-" onPress={() => updateQuantity(item.id, item.quantity -
1)} />
      </View>
    ))}
    <Text>Total: ${getTotalPrice()}</Text>
    <Button title="Clear Cart" onPress={clearCart} />
  </View>
);
}

```

```

// App.js - Using CartContext
import { CartProvider } from './contexts/CartContext';
import ProductCard from './components/ProductCard';
import CartScreen from './screens/CartScreen';

export default function App() {
  return (
    <CartProvider>
      <ProductCard product={{ id: 1, name: 'Product 1', price: 10 }} />
      <CartScreen />
    </CartProvider>
  );
}

```

Example 5: User Profile Information Context

```

// contexts/ProfileContext.js
import { createContext, useContext, useState, useEffect } from 'react';

const ProfileContext = createContext();

export const useProfile = () => {
  const context = useContext(ProfileContext);
  if (!context) {
    throw new Error('useProfile must be used within ProfileContext.Provider');
  }
  return context;
};

export const ProfileProvider = ({ children }) => {
  const [profile, setProfile] = useState(null);

```

```

    const [isLoading, setIsLoading] = useState(true);

    useEffect(() => {
        setIsLoading(false);
    }, []);

    const updateProfile = (updates) => {
        const updatedProfile = { ...profile, ...updates };
        setProfile(updatedProfile);
        return { success: true };
    };

    const value = {
        profile,
        updateProfile,
        isLoading,
    };

    return <ProfileContext.Provider value={value}>{children}
</ProfileContext.Provider>;
};

// Usage in component
function ProfileScreen() {
    const { profile, updateProfile } = useProfile();
    const [name, setName] = useState(profile?.name || '');

    const handleSave = () => {
        const result = updateProfile({ name });
        if (result.success) {
            console.log('Profile updated!');
        }
    };

    return (
        <View>
            <TextInput value={name} onChangeText={setName} placeholder="Name" />
            <Button title="Save" onPress={handleSave} />
        </View>
    );
}

```

```

// App.js - Using ProfileContext
import { ProfileProvider } from '../contexts/ProfileContext';
import ProfileScreen from '../screens/ProfileScreen';

export default function App() {
    return (
        <ProfileProvider>
            <ProfileScreen />
        </ProfileProvider>
    );
}

```

```
    );  
  }
```

3. Data Passing with Context

Context allows you to pass data between screens and components without prop drilling or navigation params.

Passing Data Between Screens

```
// contexts/DataContext.js  
import { createContext, useContext, useState } from 'react';  
  
const DataContext = createContext();  
  
export const useData = () => {  
  const context = useContext(DataContext);  
  if (!context) {  
    throw new Error('useData must be used within DataContext.Provider');  
  }  
  return context;  
};  
  
export const DataProvider = ({ children }) => {  
  const [sharedData, setSharedData] = useState(null);  
  
  return (  
    <DataContext.Provider value={{ sharedData, setSharedData }}>  
      {children}  
    </DataContext.Provider>  
  );  
};
```

```
// App.js  
import { NavigationContainer } from '@react-navigation/native';  
import { createNativeStackNavigator } from '@react-navigation/native-stack';  
import { DataProvider } from '../contexts/DataContext';  
import HomeScreen from '../screens/HomeScreen';  
import DetailsScreen from '../screens/DetailsScreen';  
  
const Stack = createNativeStackNavigator();  
  
export default function App() {  
  return (  
    <DataProvider>  
      <NavigationContainer>  
        <Stack.Navigator>  
          <Stack.Screen name="Home" component={HomeScreen} />  
          <Stack.Screen name="Details" component={DetailsScreen} />  
        </Stack.Navigator>  
      </NavigationContainer>  
    </DataProvider>  
  );  
}
```

```
        </Stack.Navigator>
      </NavigationContainer>
    </DataProvider>
  );
}
```

```
// screens/HomeScreen.js
import { View, Button } from 'react-native';
import { useData } from '../contexts/DataContext';

export default function HomeScreen({ navigation }) {
  const { setSharedData } = useData();

  const handleNavigate = () => {
    setSharedData({ message: 'Hello from Home!', timestamp: Date.now() });
    navigation.navigate('Details');
  };

  return (
    <View>
      <Button title="Go to Details" onPress={handleNavigate} />
    </View>
  );
}
```

```
// screens/DetailsScreen.js
import { View, Text } from 'react-native';
import { useData } from '../contexts/DataContext';

export default function DetailsScreen() {
  const { sharedData } = useData();

  return (
    <View>
      <Text>{sharedData?.message}</Text>
      <Text>Time: {new Date(sharedData?.timestamp).toLocaleString()}</Text>
    </View>
  );
}
```

Updating Context from Child Components

```
// contexts/UserContext.js
import { createContext, useContext, useState } from 'react';

const UserContext = createContext();
```

```
export const useUser = () => {
  const context = useContext(UserContext);
  if (!context) {
    throw new Error('useUser must be used within UserContext.Provider');
  }
  return context;
};

export const UserProvider = ({ children }) => {
  const [userData, setUserData] = useState({ name: '', email: '' });

  return (
    <UserContext.Provider value={{ userData, setUserData }}>
      {children}
    </UserContext.Provider>
  );
};
```

```
// App.js
import { UserProvider } from './contexts/UserContext';
import ProfileForm from './components/ProfileForm';
import DisplayProfile from './components/DisplayProfile';

export default function App() {
  return (
    <UserProvider>
      <ProfileForm />
      <DisplayProfile />
    </UserProvider>
  );
}
```

```
// components/ProfileForm.js
import { useState } from 'react';
import { View, TextInput, Button } from 'react-native';
import { useUser } from '../contexts/UserContext';

export default function ProfileForm() {
  const { setUserData } = useUser();
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');

  const handleSubmit = () => {
    setUserData({ name, email });
  };

  return (
    <View>
      <TextInput value={name} onChangeText={setName} placeholder="Name" />

```

```

        <TextInput value={email} onChangeText={setEmail} placeholder="Email" />
        <Button title="Save" onPress={handleSubmit} />
      </View>
    );
  }

```

```

// components/DisplayProfile.js
import { View, Text } from 'react-native';
import { useUser } from '../contexts/UserContext';

export default function DisplayProfile() {
  const { userData } = useUser();

  return (
    <View>
      <Text>Name: {userData.name}</Text>
      <Text>Email: {userData.email}</Text>
    </View>
  );
}

```

4. Brief Introduction to Redux

Redux is a predictable state container for JavaScript apps. It's an alternative to Context API for managing global state.

Why Redux?

- **Centralized State:** All state in one place
- **Predictable Updates:** State changes follow strict patterns
- **Time Travel Debugging:** Redux DevTools allow you to replay actions
- **Large Apps:** Better for complex applications with lots of state

Redux vs Context API

Feature	Context API	Redux
Learning Curve	Easy	Moderate
Boilerplate	Minimal	More code needed
DevTools	Limited	Excellent
Performance	Good for small apps	Optimized for large apps
Use Case	Simple to moderate apps	Complex apps

Basic Redux Setup

Step 1: Install Redux

```
npm install @reduxjs/toolkit react-redux
```

Step 2: Create a Store

```
// store/store.js
import { configureStore } from '@reduxjs/toolkit';
import userReducer from './slices/userSlice';
import cartReducer from './slices/cartSlice';

export const store = configureStore({
  reducer: {
    user: userReducer,
    cart: cartReducer,
  },
});
```

Step 3: Create a Slice

```
// store/slices/userSlice.js
import { createSlice } from '@reduxjs/toolkit';

const userSlice = createSlice({
  name: 'user',
  initialState: {
    user: null,
    isAuthenticated: false,
  },
  reducers: {
    login: (state, action) => {
      state.user = action.payload;
      state.isAuthenticated = true;
    },
    logout: (state) => {
      state.user = null;
      state.isAuthenticated = false;
    },
  },
});

export const { login, logout } = userSlice.actions;
export default userSlice.reducer;
```

Step 3b: Create Cart Slice

```
// store/slices/cartSlice.js
import { createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: {
    items: [],
  },
  reducers: {
    addItem: (state, action) => {
      const existingItem = state.items.find(
        (item) => item.id === action.payload.id
      );
      if (existingItem) {
        existingItem.quantity += 1;
      } else {
        state.items.push({ ...action.payload, quantity: 1 });
      }
    },
    removeItem: (state, action) => {
      state.items = state.items.filter((item) => item.id !== action.payload);
    },
    updateQuantity: (state, action) => {
      const { id, quantity } = action.payload;
      const item = state.items.find((item) => item.id === id);
      if (item) {
        if (quantity <= 0) {
          state.items = state.items.filter((item) => item.id !== id);
        } else {
          item.quantity = quantity;
        }
      }
    },
    clearCart: (state) => {
      state.items = [];
    },
  },
});

export const { addItem, removeItem, updateQuantity, clearCart } =
  cartSlice.actions;
export default cartSlice.reducer;
```

Step 4: Provide Store to App

```
// App.js
import { Provider } from 'react-redux';
import { store } from './store/store';

export default function App() {
```

```
    return (
      <Provider store={store}>
        { /* Your app components */ }
      </Provider>
    );
  }
```

Step 5: Use Redux in Components

```
// components/LoginScreen.js
import { useDispatch, useSelector } from 'react-redux';
import { login } from '../store/slices/userSlice';

function LoginScreen() {
  const dispatch = useDispatch();
  const user = useSelector((state) => state.user.user);

  const handleLogin = () => {
    dispatch(login({ id: 1, name: 'John Doe', email: 'john@example.com' }));
  };

  return (
    <View>
      <Text>User: {user?.name || 'Not logged in'}</Text>
      <Button title="Login" onPress={handleLogin} />
    </View>
  );
}
```

```
// components/CartScreen.js
import { useDispatch, useSelector } from 'react-redux';
import { addItem, removeItem, updateQuantity, clearCart } from
  '../store/slices/cartSlice';

function CartScreen() {
  const dispatch = useDispatch();
  const items = useSelector((state) => state.cart.items);

  const getTotalPrice = () => {
    return items.reduce((total, item) => total + item.price * item.quantity, 0);
  };

  return (
    <View>
      {items.map((item) => (
        <View key={item.id}>
          <Text>{item.name}</Text>
          <Text>Quantity: {item.quantity}</Text>
          <Text>Price: ${item.price * item.quantity}</Text>
        </View>
      ))}
    </View>
  );
}
```

```

        <Button
          title="Remove"
          onPress={() => dispatch(removeItem(item.id))}
        />
        <Button
          title="+"
          onPress={() =>
            dispatch(updateQuantity({ id: item.id, quantity: item.quantity + 1
          )))
        }
      />
      <Button
        title="-"
        onPress={() =>
          dispatch(updateQuantity({ id: item.id, quantity: item.quantity - 1
        )))
      }
    />
  </View>
  )))
  <Text>Total: ${getTotalPrice()}</Text>
  <Button title="Clear Cart" onPress={() => dispatch(clearCart())} />
</View>
);
}

```

```

// components/ProductCard.js
import { useDispatch } from 'react-redux';
import { addItem } from '../store/slices/cartSlice';

function ProductCard({ product }) {
  const dispatch = useDispatch();

  return (
    <View>
      <Text>{product.name}</Text>
      <Text>${product.price}</Text>
      <Button
        title="Add to Cart"
        onPress={() => dispatch(addItem(product))}
      />
    </View>
  );
}

```

```

// App.js - Complete Redux Example with Both Slices
import { Provider } from 'react-redux';
import { store } from '../store/store';
import LoginScreen from '../components/LoginScreen';

```

```
import CartScreen from './components/CartScreen';
import ProductCard from './components/ProductCard';

export default function App() {
  return (
    <Provider store={store}>
      <LoginScreen />
      <ProductCard product={{ id: 1, name: 'Product 1', price: 10 }} />
      <CartScreen />
    </Provider>
  );
}
```

When to Use Redux vs Context

Use Context API when:

- Small to medium apps
- Simple state management needs
- Few global state updates
- Team prefers less boilerplate

Use Redux when:

- Large, complex applications
- Need time-travel debugging
- Complex state logic
- Multiple developers working on state management

5. Platform-Specific Code

React Native allows you to write platform-specific code for iOS and Android.

Using Platform API

```
import { Platform, StyleSheet, View, Text } from 'react-native';

function MyComponent() {
  return (
    <View style={styles.container}>
      <Text style={styles.text}>
        Running on {Platform.OS}
      </Text>
      {Platform.OS === 'ios' && (
        <Text>This is iOS specific content</Text>
      )}
      {Platform.OS === 'android' && (
        <Text>This is Android specific content</Text>
      )}
    </View>
  );
}
```

```

    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    paddingTop: Platform.OS === 'ios' ? 20 : 0,
    paddingBottom: Platform.OS === 'android' ? 10 : 0,
  },
  text: {
    fontSize: Platform.OS === 'ios' ? 18 : 16,
    fontFamily: Platform.select({
      ios: 'Arial',
      android: 'Roboto',
    }),
  },
});

```

Platform-Specific Files

Create separate files for iOS and Android:

```

components/
  Button.ios.js    # iOS specific
  Button.android.js # Android specific
  Button.js        # Default (web)

```

```

// Button.ios.js
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

export default function Button({ title, onPress }) {
  return (
    <TouchableOpacity style={styles.iosButton} onPress={onPress}>
      <Text style={styles.iosText}>{title}</Text>
    </TouchableOpacity>
  );
}

const styles = StyleSheet.create({
  iosButton: {
    backgroundColor: '#007AFF',
    padding: 12,
    borderRadius: 8,
  },
  iosText: {
    color: 'FFFFFF',
    fontSize: 16,
  },
});

```

```
// Button.android.js
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

export default function Button({ title, onPress }) {
  return (
    <TouchableOpacity style={styles.androidButton} onPress={onPress}>
      <Text style={styles.androidText}>{title}</Text>
    </TouchableOpacity>
  );
}

const styles = StyleSheet.create({
  androidButton: {
    backgroundColor: '#6200EE',
    padding: 12,
    borderRadius: 4,
    elevation: 2,
  },
  androidText: {
    color: '#FFFFFF',
    fontSize: 14,
    textTransform: 'uppercase',
  },
});
```

Platform-Specific Styles

```
import { Platform, StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  container: {
    ...Platform.select({
      ios: {
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.25,
        shadowRadius: 3.84,
      },
      android: {
        elevation: 5,
      },
    }),
  },
  button: {
    padding: Platform.select({
      ios: 12,
      android: 10,
      default: 8,
    }),
  },
});
```

```
    },  
  });  
});
```

Platform Version Detection

```
import { Platform } from 'react-native';  
  
if (Platform.OS === 'ios') {  
  if (Platform.Version >= 13) {  
    // iOS 13+ specific code  
  }  
} else if (Platform.OS === 'android') {  
  if (Platform.Version >= 29) {  
    // Android 10+ specific code  
  }  
}
```

App.js Example with Platform-Specific Code

```
// App.js  
import { View, StyleSheet } from 'react-native';  
import { Platform } from 'react-native';  
import MyComponent from './components/MyComponent';  
  
export default function App() {  
  return (  
    <View style={styles.container}>  
      <MyComponent />  
    </View>  
  );  
}  
  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    paddingTop: Platform.OS === 'ios' ? 20 : 0,  
    backgroundColor: Platform.select({  
      ios: '#F5F5F5',  
      android: '#FFFFFF',  
    }),  
  },  
});
```

6. Combining All Contexts - Complete Example

In real-world applications, you'll often need to combine multiple contexts. Here's a complete example showing how to use all contexts together in App.js:

```
// App.js - Complete Example with All Contexts
import { AuthProvider } from './contexts/AuthContext';
import { ThemeProvider } from './contexts/ThemeContext';
import { LanguageProvider } from './contexts/LanguageContext';
import { CartProvider } from './contexts/CartContext';
import { ProfileProvider } from './contexts/ProfileContext';
import { DataProvider } from './contexts/DataContext';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import HomeScreen from './screens/HomeScreen';
import ProfileScreen from './screens/ProfileScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <ThemeProvider>
      <LanguageProvider>
        <AuthProvider>
          <ProfileProvider>
            <CartProvider>
              <DataProvider>
                <NavigationContainer>
                  <Stack.Navigator>
                    <Stack.Screen name="Home" component={HomeScreen} />
                    <Stack.Screen name="Profile" component={ProfileScreen} />
                  </Stack.Navigator>
                </NavigationContainer>
              </DataProvider>
            </CartProvider>
          </ProfileProvider>
        </AuthProvider>
      </LanguageProvider>
    </ThemeProvider>
  );
}
```

Using Multiple Contexts in Components

```
// components/Header.js
import { View, Text, StyleSheet } from 'react-native';
import { useAuth } from '../contexts/AuthContext';
import { useTheme } from '../contexts/ThemeContext';
import { useLanguage } from '../contexts/LanguageContext';
import { useCart } from '../contexts/CartContext';

export default function Header() {
```

```

const { user } = useAuth();
const { colors } = useTheme();
const { t } = useLanguage();
const { getTotalItems } = useCart();

return (
  <View style={[styles.header, { backgroundColor: colors.background }]}>
    <Text style={[styles.text, { color: colors.text }]}>
      {t('welcome')}, {user?.name || 'Guest'}
    </Text>
    <Text style={[styles.text, { color: colors.text }]}>
      Cart: {getTotalItems()} items
    </Text>
  </View>
);
}

const styles = StyleSheet.create({
  header: {
    padding: 15,
    borderBottomWidth: 1,
    borderBottomColor: '#e0e0e0',
  },
  text: {
    fontSize: 16,
    marginBottom: 5,
  },
});

```

```

// screens/HomeScreen.js
import { View, Text, Button, StyleSheet } from 'react-native';
import { useAuth } from '../contexts/AuthContext';
import { useTheme } from '../contexts/ThemeContext';
import { useLanguage } from '../contexts/LanguageContext';
import { useCart } from '../contexts/CartContext';
import Header from '../components/Header';

export default function HomeScreen({ navigation }) {
  const { user, login, logout, isAuthenticated } = useAuth();
  const { colors, toggleTheme } = useTheme();
  const { t, changeLanguage } = useLanguage();
  const { addItem, getTotalItems } = useCart();

  const handleAddToCart = () => {
    addItem({ id: 1, name: 'Sample Product', price: 10 });
  };

  return (
    <View style={[styles.container, { backgroundColor: colors.background }]}>
      <Header />
      <Text style={[styles.title, { color: colors.text }]}>

```

```

        {t('welcome')}}
      </Text>
      {isAuthenticated ? (
        <View>
          <Text style={{ color: colors.text }}>
            {t('welcome')}, {user?.name}
          </Text>
          <Button title={t('logout')} onPress={logout} />
        </View>
      ) : (
        <Button title={t('login')} onPress={() => login('test@example.com',
'password')} />
      )}
      <Button title="Toggle Theme" onPress={toggleTheme} />
      <Button title="Change Language" onPress={() => changeLanguage('es')} />
      <Button title="Add to Cart" onPress={handleAddToCart} />
      <Text style={{ color: colors.text }}>
        Cart Items: {getTotalItems()}
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
  },
});

```

Best Practices for Combining Contexts

1. **Order Matters:** Wrap providers in order of dependency (Theme → Language → Auth → Profile → Cart → Data)
2. **Keep Providers Focused:** Each provider should handle one concern
3. **Avoid Over-nesting:** If you have too many providers, consider using Redux
4. **Performance:** Context providers re-render all consumers when value changes - use `useMemo` for complex values
5. **Type Safety:** Consider using TypeScript for better type checking across contexts

7. Brief Introduction to TypeScript

TypeScript adds type safety to JavaScript, helping catch errors before runtime.

Why TypeScript?

- **Type Safety:** Catch errors during development
- **Better IDE Support:** Autocomplete and IntelliSense
- **Refactoring:** Easier to refactor large codebases
- **Documentation:** Types serve as documentation

Creating a TypeScript Expo App

Option 1: Create New TypeScript App with Expo

```
npx create-expo-app@latest MyTypeScriptApp --template blank-typescript
```

This command creates a new Expo app with TypeScript already configured.

Option 2: Add TypeScript to Existing Project

Step 1: Install TypeScript

```
npm install --save-dev typescript @types/react @types/react-native
```

Step 2: Create tsconfig.json

```
{
  "compilerOptions": {
    "target": "esnext",
    "module": "commonjs",
    "lib": ["es2017"],
    "jsx": "react-native",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "resolveJsonModule": true
  },
  "exclude": ["node_modules"]
}
```

Basic TypeScript Examples

Typing Components

```
// components/Button.tsx
import React from 'react';
import { TouchableOpacity, Text, StyleSheet } from 'react-native';

interface ButtonProps {
```

```
    title: string;
    onPress: () => void;
    disabled?: boolean;
  }

export default function Button({ title, onPress, disabled = false }: ButtonProps)
{
  return (
    <TouchableOpacity
      style={[styles.button, disabled && styles.disabled]}
      onPress={onPress}
      disabled={disabled}
    >
      <Text style={styles.text}>{title}</Text>
    </TouchableOpacity>
  );
}

const styles = StyleSheet.create({
  button: {
    backgroundColor: '#007AFF',
    padding: 12,
    borderRadius: 8,
  },
  disabled: {
    opacity: 0.5,
  },
  text: {
    color: '#FFFFFF',
    fontSize: 16,
  },
});
```

Typing Context

```
// contexts/AuthContext.tsx
import { createContext, useContext, useState, ReactNode } from 'react';

interface User {
  id: number;
  name: string;
  email: string;
}

interface AuthContextType {
  user: User | null;
  login: (email: string, password: string) => Promise<boolean>;
  logout: () => void;
  isAuthenticated: boolean;
}
```

```

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within AuthProvider');
  }
  return context;
};

interface AuthProviderProps {
  children: ReactNode;
}

export const AuthProvider = ({ children }: AuthProviderProps) => {
  const [user, setUser] = useState<User | null>(null);

  const login = async (email: string, password: string): Promise<boolean> => {
    // Login logic
    return true;
  };

  const logout = () => {
    setUser(null);
  };

  const value: AuthContextType = {
    user,
    login,
    logout,
    isAuthenticated: !!user,
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
};

```

Typing Navigation

```

// types/navigation.ts
import { NavigatorScreenParams } from '@react-navigation/native';

export type RootStackParamList = {
  Login: undefined;
  Home: undefined;
  Profile: { userId: number };
  Settings: undefined;
};

export type TabParamList = {
  Home: undefined;
  Search: undefined;
};

```

```
Profile: NavigatorScreenParams<ProfileStackParamList>;  
};  
  
export type ProfileStackParamList = {  
  Profile: { userId: number };  
  EditProfile: { userId: number };  
};
```

```
// screens/ProfileScreen.tsx  
import { StackScreenProps } from '@react-navigation/stack';  
import { RootStackParamList } from '../types/navigation';  
  
type Props = StackScreenProps<RootStackParamList, 'Profile'>;  
  
export default function ProfileScreen({ route, navigation }: Props) {  
  const { userId } = route.params;  
  
  return (  
    <View>  
      <Text>User ID: {userId}</Text>  
    </View>  
  );  
}
```

Common TypeScript Patterns

```
// Union types  
type Theme = 'light' | 'dark';  
  
// Optional properties  
interface User {  
  id: number;  
  name: string;  
  email?: string; // Optional  
}  
  
// Function types  
type OnPress = () => void;  
type OnChange = (value: string) => void;  
  
// Generic types  
interface ApiResponse<T> {  
  data: T;  
  status: number;  
}  
  
// Type assertions  
const value = someValue as string;
```

8. React Native DevTools

React Native provides several tools for debugging and development.

React Native Debugger

Installation

```
# Download from: https://github.com/jhen0409/react-native-debugger
# Or use Expo DevTools (built-in)
```

Features

- **React DevTools:** Inspect component tree and props
- **Redux DevTools:** Debug Redux state (if using Redux)
- **Network Inspector:** Monitor API calls
- **Console:** View logs and errors

Using React DevTools

1. **Start your app:** `npx expo start`
2. **Open DevTools:** Shake device or press `Cmd+D` (iOS) / `Cmd+M` (Android)
3. **Select "Debug":** Opens Chrome DevTools
4. **Inspect Components:** Use React DevTools extension

Console Logging

```
// Basic logging
console.log('Debug message');
console.warn('Warning message');
console.error('Error message');

// Object inspection
console.log('User:', user);
console.table(items); // Display as table

// Grouped logs
console.group('API Calls');
console.log('Request 1');
console.log('Request 2');
console.groupEnd();
```

Summary

This tutorial covered advanced React Native concepts:

1. **React Context API:** Sharing state across components without prop drilling
2. **Nested Contexts:** Multiple contexts for different app concerns (Auth, Theme, Language, Cart, Profile) with individual App.js examples
3. **Data Passing with Context:** Sharing data between screens and components
4. **Brief Introduction to Redux:** Alternative state management solution
5. **Platform-Specific Code:** Handling iOS and Android differences
6. **Combining All Contexts:** Complete example showing how to use all contexts together in a single App.js
7. **Brief Introduction to TypeScript:** Type safety in React Native
8. **React Native DevTools:** Debugging and development tools

Next Steps

- Practice building apps with multiple contexts
- Add TypeScript to your projects
- Explore Redux for complex state management
- Master React Native DevTools for debugging
- Create platform-specific components for iOS and Android

Additional Resources

- [React Context Documentation](#)
 - [Redux Toolkit Documentation](#)
 - [TypeScript Handbook](#)
 - [React Native Debugging](#)
 - [Flipper Documentation](#)
-

Practice Home

1. Implement Cart or Language Provider

Choose one and implement it in a project:

- **Cart:** Create `CartContext` with add/remove/update functions, build product listing and cart screens, display totals in header
- **Language:** Create `LanguageContext` with translation support for 2+ languages, implement language switching, translate UI text

Requirements: Separate context file, use provider in `App.js`, access context in 3+ components (see Section 2 for patterns)

2. Use Redux Where Applicable

Practice Redux for complex state scenarios:

- Install: `npm install @reduxjs/toolkit react-redux`
- Create store with 2+ slices (e.g., user, cart)
- Use `useSelector` and `useDispatch` in components
- Compare with Context API implementation

Use Redux when: Large apps, complex state logic, need time-travel debugging, multiple developers

Use Context when: Small-medium apps, simple state, prefer less boilerplate

3. Practice TypeScript for Type Safety

For beginners interested in type safety:

```
npx create-expo-app@latest MyTypeScriptApp --template blank-typescript
```

Tasks: Convert 3+ components to `.tsx` with prop interfaces, convert 1 context with typed values, add navigation types using `StackScreenProps`

Example:

```
interface ButtonProps {  
  title: string;  
  onPress: () => void;  
  disabled?: boolean;  
}  
  
export default function Button({ title, onPress, disabled = false }: ButtonProps)  
{  
  return <TouchableOpacity onPress={onPress} disabled={disabled}>  
    <Text>{title}</Text>  
  </TouchableOpacity>;  
}
```

Benefits: Catch errors early, better IDE support, self-documenting code, easier refactoring